

ChatApp System Architecture & Overview

An enterprise-grade, event-driven messaging application built with Express, TypeScript, RabbitMQ, Redis, MongoDB, and Next.js.

ARCHITECTURE

Decoupled Microservices

FRONTEND

Next.js 16 (App Router)

MESSAGE BROKER

RabbitMQ (AMQP)

DATABASE & CACHE

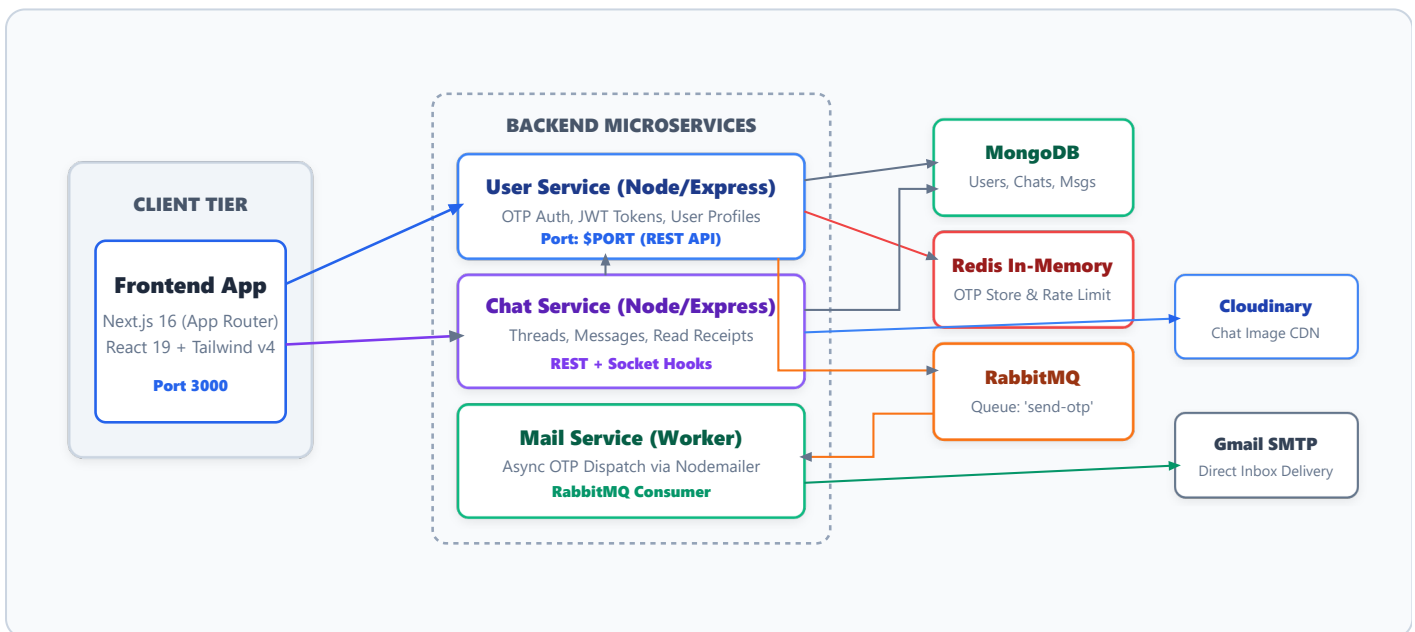
MongoDB + Redis

01 Executive Summary

ChatApp is a full-stack, distributed chat platform designed for reliable messaging, instant media exchange, and secure passwordless authentication. The backend is partitioned into independent microservices communicating synchronously via REST and asynchronously through RabbitMQ message queues.

This architecture decouples CPU- and I/O-intensive operations (such as email delivery and media uploads) from core API request paths, providing high availability, low request latency, and granular scalability.

02 Architectural Topology



03 Microservices Deep Dive

User Service

backend/user

Authentication & Identity Provider

- Passwordless email login via 6-digit OTP codes.
- Redis rate limiting (60s cooldown per email) to stop spam.
- Automatic user provisioning upon first successful OTP verification.
- Issues HMAC-SHA256 JWT tokens with 15-day expiration.
- Publishes background events to RabbitMQ.

Chat Service

backend/chat

Conversation & Messaging Core

- 1-on-1 thread creation and duplicate-prevention index.
- Unseen message count aggregation per conversation.
- Cloudinary storage integration via Multer for photos/attachments.
- Inter-service user hydration from User Service via Axios.
- Automatic read receipts (marking seen/seenAt timestamp).

Mail Service

backend/mail

Async Notification Worker

- Event-driven worker consuming from RabbitMQ queue `send-otp`.
- Nodemailer SMTP integration for guaranteed delivery.
- Protects the core API from email latency (1-3 sec saved per login).
- Explicit message acknowledgments (`channel.ack`) prevent message loss.

04 API Specification & Routes

1. User Service Routes (/api/v1)

Method	Endpoint	Auth Required	Description & Actions
POST	/login	No	Checks rate limit in Redis, generates 6-digit OTP, stores in Redis (5 min TTL), publishes to RabbitMQ <code>send-otp</code> .
POST	/verify	No	Validates submitted OTP with Redis. Creates user in Mongo if first time. Issues and returns JWT token + user object.
GET	/me	Bearer JWT	Retrieves the authenticated user's profile details.
GET	/user/all	Bearer JWT	Lists all registered users in the platform for starting new conversations.
GET	/user/:id	No	Lookup user profile by ID. Used internally by the Chat Service to hydrate participant metadata.
POST	/update/user	Bearer JWT	Updates the authenticated user's display name and re-issues a fresh JWT token.

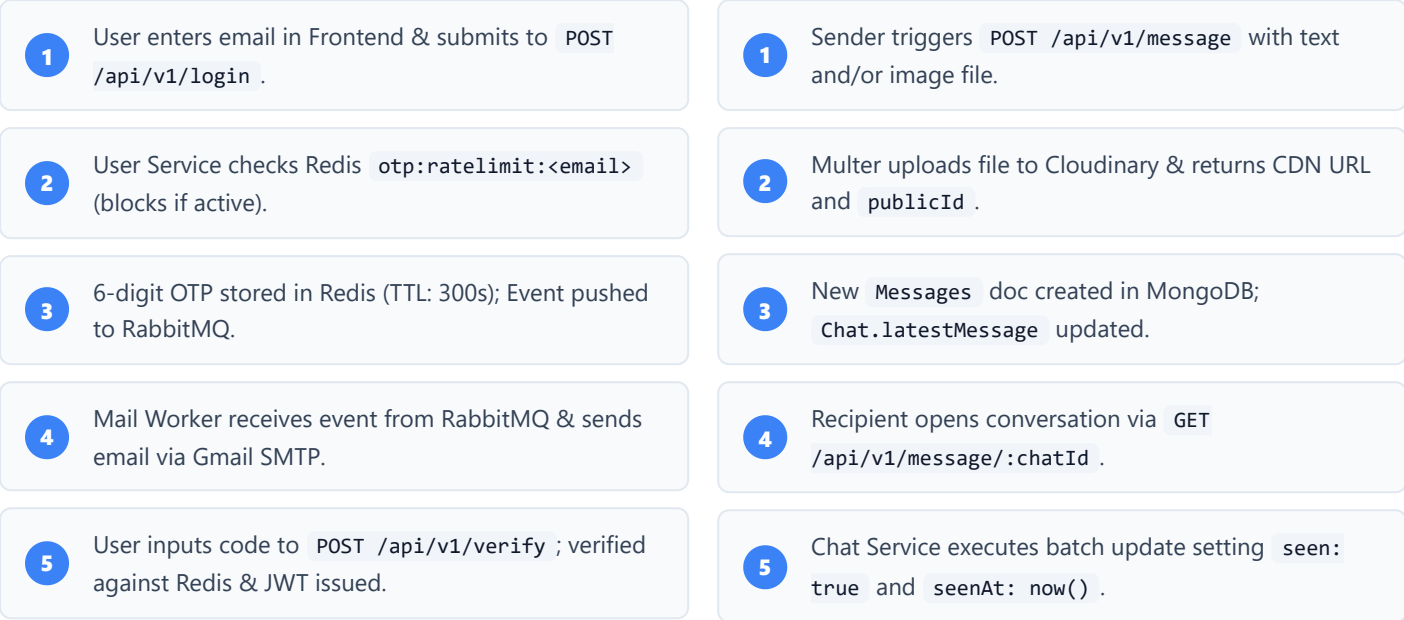
2. Chat Service Routes (/api/v1)

Method	Endpoint	Auth Required	Description & Actions
POST	/chat/new	Bearer JWT	Finds existing chat between the 2 users or instantiates a new <code>Chat</code> document.
GET	/chat/all	Bearer JWT	Fetches all user chats sorted by last activity, calculates unread counts, and hydrates the other user's profile via REST.
POST	/message	Bearer JWT	Uploads attachment to Cloudinary (if present), stores message in MongoDB, and updates conversation's <code>latestMessage</code> .
GET	/message/:chatId	Bearer JWT	Fetches message history for a conversation and marks unread incoming messages as <code>seen: true</code> with timestamp.

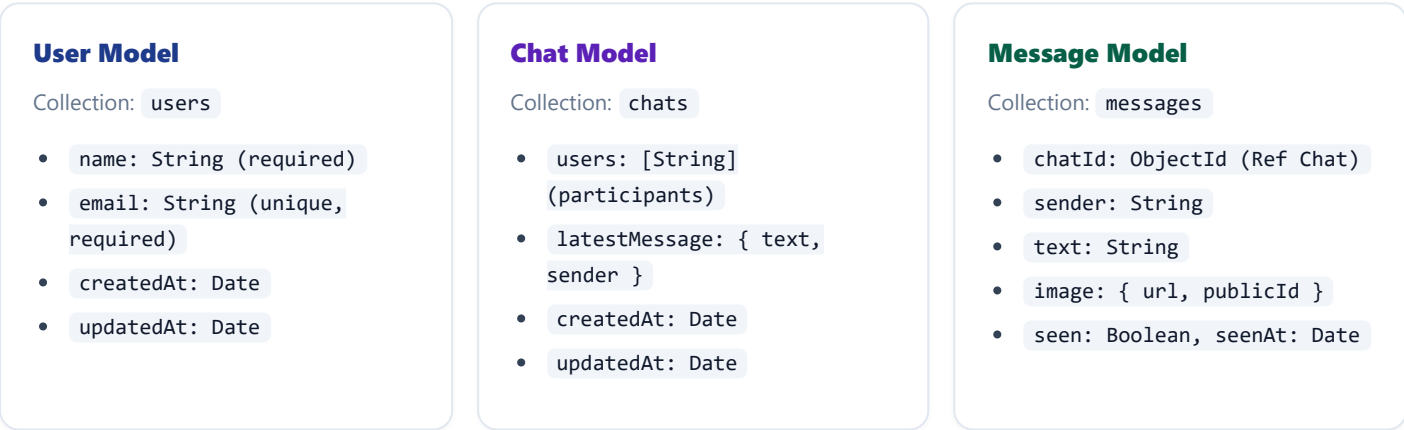
05 Key Lifecycle Sequences

Authentication & OTP Sequence

Chat & Messaging Sequence



06 Database Schema Models



07 Roadmap & Production Recommendations

